

LLM Exploration

Naseebullah Andar, Samantha Agisim, Patrick Ledoit

1 LLM Exploration Logs

This section documents our use of LLMs throughout the project in a chronological, log-style format. The goal is to make clear where LLMs helped with ideation, code scaffolding, debugging, data inspection, and writing, and where we exercised our own judgment. In several phases, LLMs produced draft code or prose that accelerated iteration, but we remained responsible for deciding the research direction, validating assumptions, running experiments, and determining what to keep in the final project.

1.1 Phase 1: Project Ideation and Whiteboard Framing

LLM Use:

- Asked exploratory questions about convex mean-variance optimization, long-only versus long-short constraints, and possible instruments such as equities, ETFs, commodities, crypto, and later prediction markets.
- Used LLMs to help convert broad whiteboard brainstorming into a staged research plan with a clearer mathematical focus.
- Discussed whether the project should be framed primarily as return prediction or portfolio optimization.

Our Work:

- Evaluated candidate directions and selected dynamic portfolio optimization as the main focus.
- Chose to begin with a mean-variance style framework with turnover penalties rather than a pure prediction task.
- Identified realistic constraints and benchmark ideas early in the process.

Outcome: The project moved from broad brainstorming to a more precise optimization-based research question.

1.2 Phase 2: Data Pipeline and Model Setup

LLM Use:

- Requested help structuring code for loading and cleaning equity data.
- Asked for guidance on organizing return panels, rolling windows, and lookahead-safe feature construction.
- Used LLM-generated code scaffolding for flexible data loading and feature extraction.

Our Work:

- Built the return panel from raw data.
- Implemented filtering such as exchange codes, share codes, price filters, and data sufficiency checks.
- Constructed rolling estimation windows and validated the dataset structure.

Outcome: A functional data pipeline was established for dynamic portfolio experiments.

1.3 Phase 3: Optimization Implementation

LLM Use:

- Asked how to implement constrained optimization using softmax and related parameterizations.
- Used LLMs to debug PyTorch gradient updates and feasibility issues.
- Asked for suggestions on numerically stable ways to maintain valid weights.

Our Work:

- Implemented the objective function

$$\mu_t^\top w - \gamma w^\top \Sigma_t w - \kappa \|w - w_{t-1}\|_1.$$

- Built the full optimization loop with gradient-based updates.
- Ensured that the weights satisfied the required portfolio constraints.

Outcome: A working dynamic optimizer was integrated into the backtesting framework.

1.4 Phase 4: Sanity Checks

LLM Use:

- Asked whether simple toy experiments were sufficient for validating the optimizer.
- Requested ideas for edge cases such as singular covariance matrices, zero features, and overly tight constraints.

Our Work:

- Designed two-asset and other toy experiments.

- Verified allocation behavior manually and checked whether the optimizer behaved in economically intuitive ways.
- Used these checks to catch implementation mistakes before larger-scale runs.

Outcome: The optimizer passed basic validation checks and behaved as expected in controlled settings.

1.5 Phase 5: Hyperparameter Exploration

LLM Use:

- Requested code for grid search, plotting, and summary metrics.
- Asked how to interpret unstable performance across parameter choices.

Our Work:

- Ran experiments across rolling windows, learning rates, gradient steps, and penalty parameters.
- Generated cumulative return plots and performance summaries.
- Compared sensitivity across specifications instead of relying on a single tuned setting.

Outcome: We identified that performance was sensitive to the optimization setup and that instability itself was an important finding.

1.6 Phase 6: Backtesting and Benchmarking

LLM Use:

- Asked how to structure performance metrics and benchmark comparisons.
- Requested help formatting results tables and summaries.

Our Work:

- Built the full backtest pipeline.
- Compared the strategy against equal-weight and other simple baselines.
- Computed Sharpe ratios, volatility, drawdowns, and related performance statistics.

Outcome: A complete evaluation framework was created for comparing portfolio performance.

1.7 Phase 7: Debugging and Pipeline Hardening

LLM Use:

- Diagnosed coding errors such as missing variables, plotting issues, and inconsistent data storage.
- Suggested ways to improve reproducibility and modularity.

Our Work:

- Fixed the underlying data structures and corrected the experiment pipeline.
- Re-ran experiments after each correction and checked whether outputs were consistent.
- Used the debugging process to make the codebase more robust.

Outcome: The pipeline became more reproducible and less fragile under repeated experimentation.

1.8 Phase 8: Daily Data Extension

LLM Use:

- Asked how to adapt the monthly framework to daily data without changing the core model logic.

- Requested code restructuring that would preserve no-leakage principles.

Our Work:

- Implemented a daily return panel and re-ran the optimizer at higher frequency.
- Compared the behavior of the same framework under monthly and daily inputs.
- Interpreted the effect of increased noise and turnover at higher frequency.

Outcome: We identified important limits of the model when moving to noisier daily data.

1.9 Phase 9: Project Refinement and Differentiation

LLM Use:

- Worked through `finding_project_ideas.md` and `project_refinement.md` to sharpen the project statement.
- Asked for help distinguishing our work from prior portfolio-optimization projects and from more generic template language.
- Used LLMs to rewrite dense bullet-heavy drafts into clearer narrative prose for the development log and self-critique.
- Explored differentiators such as skewness-aware objectives and an LLM-driven news or sentiment input.

Our Work:

- Decided how to position the project relative to classical mean-variance optimization, two-stage methods, and prior online-gradient approaches.
- Evaluated which extensions were realistic within the project timeline and which should remain future work.

- Reframed the written materials so that the project contribution was stated more clearly.

Outcome: The project narrative became more specific, less template-driven, and better differentiated from related work.

1.10 Phase 10: Pivoting from Equities to Prediction Markets

LLM Use:

- After receiving feedback that the original equity portfolio optimizer was too close to prior projects, we used LLMs to brainstorm ways to preserve the mathematical structure of the project while moving into a more original domain.
- We asked whether prediction markets could be treated as a portfolio-selection problem, where each contract has a price, a payoff, a probability estimate, liquidity constraints, and event-level exposure limits.
- LLMs helped us compare potential data sources, including Polymarket and Kalshi, and think through what would be required to turn market prices into a usable empirical dataset.

Our Work:

- We decided that prediction markets were a natural extension of the original optimizer because the core question remained similar: how should capital be allocated across risky opportunities under constraints?
- We reframed the project from “stock selection” to “mispricing-based prediction market allocation.” Instead of estimating expected stock returns, the new goal was to estimate whether a contract’s probability appeared high or low relative to its executable market price.

- We also had to narrow the scope. A full prediction-market trading system would require historical prices, resolved outcomes, and realistic execution assumptions. Since our available data was limited, we treated the prediction-market portion as a structured research extension rather than pretending it was a fully validated trading strategy.

Outcome: This phase changed the direction of the project. The optimizer was no longer just an equities tool; it became a general allocation framework that could be adapted to prediction-market contracts. This pivot also made the final project more distinct from standard mean-variance portfolio optimization.

1.11 Phase 11: Polymarket Data Exploration

LLM Use:

- We used LLMs to help inspect the Polymarket data files and understand which fields were actually useful for modeling.
- LLMs helped generate exploratory notebook code for loading `polymarket_markets_rich.csv`, checking missingness, parsing nested JSON-like columns such as `outcomePrices` and `events`, and summarizing fields related to volume, liquidity, bid-ask spreads, and market metadata.
- We also asked LLMs to explain the difference between a cross-sectional snapshot of open markets and a historical panel suitable for backtesting.

Our Work:

- We reviewed the generated exploratory code line by line and checked whether the outputs matched the structure of the actual dataset.
- We realized that the Polymarket data did not automatically give us everything we needed for a clean out-of-sample trading test. In particular, many markets were still

open, and the dataset was closer to a market snapshot than a complete history with resolved outcomes.

- This forced us to separate what we could actually test from what would require additional data. We could study pricing, liquidity, and cross-sectional signals, but we had to be careful about making claims about realized profitability.

Outcome: The main value of this phase was not just cleaning the data. It helped us understand the empirical limits of the prediction-market extension. We learned that the main challenge was not writing an optimizer, but defining a valid prediction-market experiment with the data available.

1.12 Phase 12: Building a Baseline Prediction-Market Pipeline

LLM Use:

- We asked LLMs to help translate the original portfolio-optimization code structure into a prediction-market setting.
- LLMs helped scaffold modules for feature construction, probability prediction, executable price approximation, portfolio optimization, diagnostics, and saved experiment outputs.
- We used LLMs to think through constraints that are specific to prediction markets, including cash holding, long-only contract positions, per-market caps, per-event caps, and liquidity-based position limits.
- We also used LLMs for debugging when the projection step became computationally expensive. The LLM suggested simpler cyclic projection logic that was easier to run and reason about.

Our Work:

- We adapted the original portfolio objective to a prediction-market setting. Instead of expected stock returns, the reward term became the estimated edge between our probability estimate and the executable contract price:

$$\sum_i w_i(\hat{p}_i - c_i^{\text{exec}}) - \gamma \sum_i w_i^2 \hat{p}_i(1 - \hat{p}_i) - \kappa \|w_t - w_{t-1}\|_1.$$

- We interpreted $\hat{p}_i - c_i^{\text{exec}}$ as a simple mispricing signal: if our estimated probability was above the market-implied price, the contract looked attractive; if it was below, the contract did not.
- We kept the model intentionally simple. Rather than building a complicated black-box predictor, we focused on whether a baseline signal could be organized into a transparent allocation framework.
- We checked whether the constraints made economic sense. For example, even if a contract appeared attractive, the optimizer should not put the entire portfolio into one illiquid market or one event category.

Outcome: This phase produced a modular baseline pipeline for prediction-market allocation. The pipeline was useful because it made the research question concrete: given a set of prediction-market contracts, estimated probabilities, executable prices, and liquidity constraints, how would the optimizer allocate capital?

1.13 Phase 13: Walk-Forward Splits, Rank IC, and PnL Diagnostics

LLM Use:

- We used LLMs to understand how to evaluate prediction-market signals without accidentally leaking future information.

- LLMs helped us define walk-forward splits based on market creation time and think through why random train-test splits would be inappropriate for time-sensitive market data.
- We asked LLMs to explain Rank IC and how it could be used as a diagnostic for whether our model was at least ranking opportunities in a meaningful way.
- LLMs also helped draft plotting code for fold-level diagnostics, including Rank IC by fold, PnL by fold, and performance broken down by liquidity bucket.

Our Work:

- We used walk-forward validation to make the evaluation more realistic. The model could only use information available before the test period, which better matches how a trading strategy would actually operate.
- We used Rank IC as a directional diagnostic. Instead of only asking whether the model produced profitable trades, Rank IC asks whether higher-scored contracts tended to correspond to better subsequent opportunities.
- We examined liquidity buckets because prediction-market returns are not meaningful without considering whether a position could realistically be entered or exited. A signal that only works in illiquid markets would be much less credible.
- We treated PnL results cautiously because our data limitations made it difficult to claim a fully realistic trading backtest. We used PnL as an exploratory diagnostic rather than the sole measure of success.

Outcome: This phase gave us a more defensible experimental structure. The most important improvement was moving from “we built a model” to “we tested whether the model ranked contracts consistently across time and liquidity conditions.” That made the prediction-market section more empirical and less speculative.

1.14 Phase 14: Debugging the Prediction-Market Codebase

LLM Use:

- We used LLMs frequently for debugging because the prediction-market pipeline involved many moving parts: parsed market data, feature matrices, time splits, optimization constraints, saved outputs, and plots.
- LLMs helped diagnose errors caused by missing packages, incorrect file paths, inconsistent column names, and notebook-to-script conversion issues.
- We also used LLMs to clean up code organization so that the project could be run from a smaller number of scripts instead of scattered notebook cells.

Our Work:

- We checked that each debugging suggestion actually matched our local files and repository structure before keeping it.
- We re-ran scripts after changes and compared outputs to make sure that fixes did not silently alter the experiment logic.
- We made judgment calls about which LLM suggestions were too ambitious or unnecessary for the final project timeline.

Outcome: The LLM was helpful as a debugging assistant, but it did not eliminate the need for manual verification. In several cases, the suggested code looked plausible but had to be changed because it did not match the exact files, columns, or experimental setup we were using.

1.15 Phase 15: Slides, Report Writing, and Presentation Preparation

LLM Use:

- We used LLMs to help turn technical notes into clearer slide language and speaker notes.
- LLMs helped us explain terms such as Rank IC, liquidity buckets, folds, and mispricing in language that would be understandable to an audience seeing the project for the first time.
- We also used LLMs to revise paragraphs in the final report so that the transition from the original equity optimizer to the Polymarket extension felt coherent rather than abrupt.

Our Work:

- We selected the actual figures, metrics, and experiments that appeared in the final presentation.
- We edited the generated prose to avoid overstating results. In particular, we tried to distinguish between implemented experiments, exploratory diagnostics, and future work requiring better historical data.
- We prepared answers to likely questions about why the project pivoted, why prediction markets were a reasonable extension, and what the current results do and do not prove.

Outcome: The final presentation became clearer because we were able to explain the project as one continuous idea: starting from portfolio optimization, identifying limitations in the initial equity setting, and then adapting the allocation framework to prediction markets.

1.16 Final Reflection

LLMs were useful throughout the project, but their role changed over time. Early on, they mostly helped with brainstorming, code scaffolding, and debugging the equity portfolio optimizer. Later, once the project shifted toward prediction markets, they became more

useful for helping us impose structure on a messy and unfamiliar data problem. The most important thing the LLM helped with was speed. It made it easier to generate starter code, outline possible experiments, explain unfamiliar diagnostics, and rewrite rough notes into clearer prose. This was especially helpful during the Polymarket phase, where we were dealing with nested data fields, incomplete historical information, and uncertainty about what claims the data could support. At the same time, the LLM often produced answers that sounded more complete than the evidence justified. For example, it could quickly suggest a full prediction-market backtest pipeline, but we still had to determine whether our data actually supported that backtest. It could describe PnL evaluation, but we had to recognize that unresolved markets and limited historical outcome data made PnL less conclusive. It could propose optimization constraints, but we had to decide which constraints were economically meaningful and feasible to implement. Overall, LLMs served as a technical assistant rather than an automatic research engine. They helped us move faster, but the project still depended on our own decisions: choosing the pivot, interpreting the data limitations, deciding which experiments were credible, checking generated code, and being honest about what the prediction-market results could and could not show.